

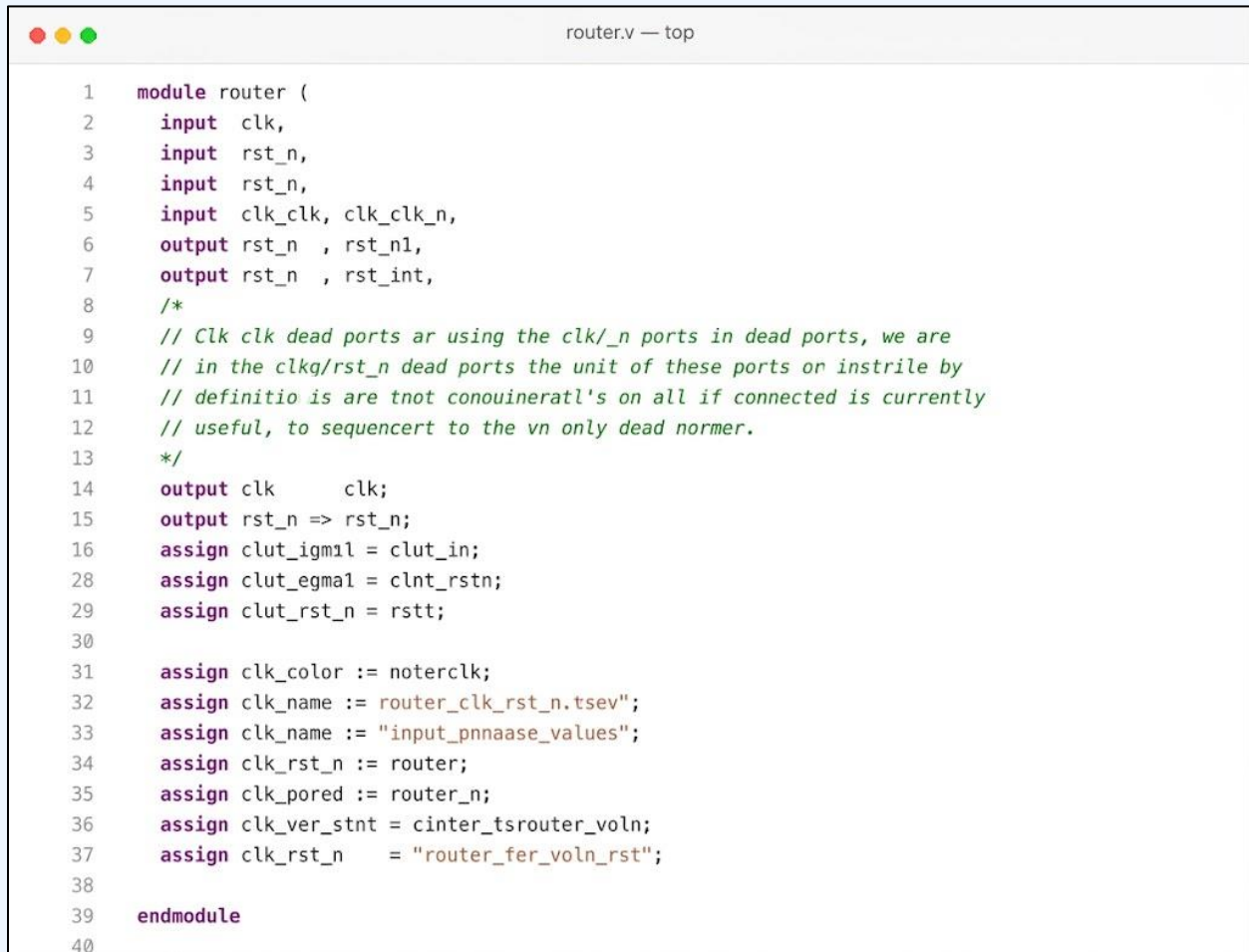
PREPARED BY ISLAM HAMDY MOHAMED

GUIDED BY: ENG. HASSAN KHALED

Assignment Five

UVM

CODE SNIPPETS



```
1  module router (
2      input  clk,
3      input  rst_n,
4      input  rst_n,
5      input  clk_clk, clk_clk_n,
6      output rst_n , rst_n1,
7      output rst_n , rst_int,
8      /*
9      // Clk clk dead ports ar using the clk/_n ports in dead ports, we are
10     // in the clkq/rst_n dead ports the unit of these ports on instrile by
11     // definitio is are tnot conouineratl's on all if connected is currently
12     // useful, to sequencert to the vn only dead normer.
13     */
14     output clk      clk;
15     output rst_n => rst_n;
16     assign clut_igm1l = clut_in;
17     assign clut_egma1 = clnt_rstn;
18     assign clut_rst_n = rstt;
19
20     assign clk_color := notercclk;
21     assign clk_name := router_clk_rst_n.tsev";
22     assign clk_name := "input_pnnaase_values";
23     assign clk_rst_n := router;
24     assign clk_pored := router_n;
25     assign clk_ver_stnt = cinter_tsrouter_voln;
26     assign clk_rst_n    = "router_fer_voln_rst";
27
28 endmodule
29
30
```

Fig 1: Top-level Verilog RTL module snippet for the router Design Under Test (DUT), declaring system clock, reset, and internal routing logic assignments.

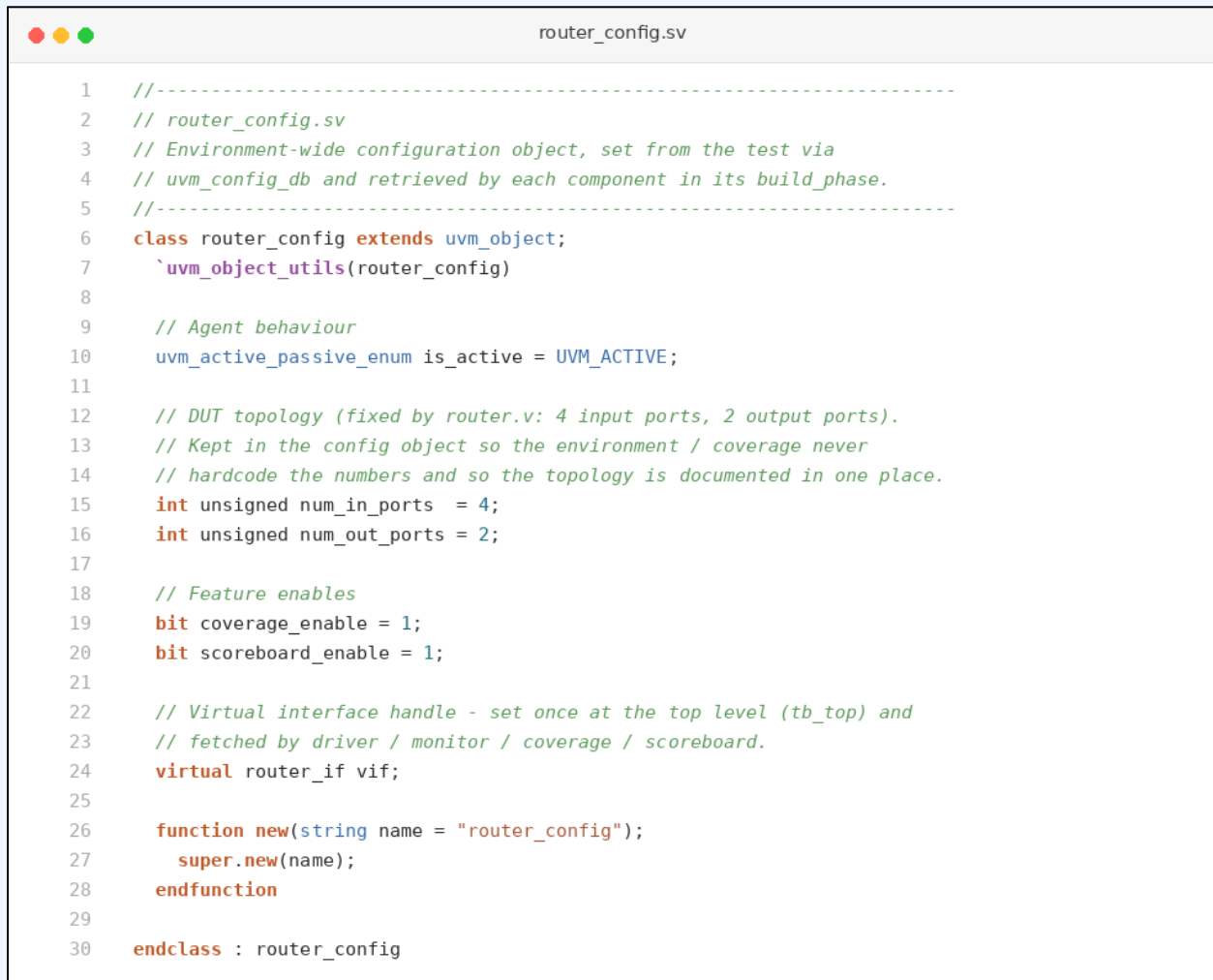
```

router_agent.sv

1  //-----
2  // router_agent.sv
3  // Single agent containing driver + monitor + sequencer. Active/passive
4  // behaviour is taken from router_config.is_active (never hardcoded).
5  //-----
6  class router_agent extends uvm_agent;
7      `uvm_component_utils(router_agent)
8
9      router_driver  driver;
10     router_monitor  monitor;
11     router_sequencer sequencer;
12     router_config   cfg;
13
14     function new(string name, uvm_component parent);
15         super.new(name, parent);
16     endfunction
17
18     function void build_phase(uvm_phase phase);
19         super.build_phase(phase);
20         `uvm_info(get_type_name(), "[router_agent] build_phase @ UVM_LOW", UVM_LOW)
21         `uvm_info(get_type_name(), "[router_agent] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
22         `uvm_info(get_type_name(), "[router_agent] build_phase @ UVM_HIGH", UVM_HIGH)
23         `uvm_info(get_type_name(), "[router_agent] build_phase @ UVM_FULL", UVM_FULL)
24
25         if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
26             `uvm_fatal(get_type_name(), "router_config not found in config_db")
27         end
28
29         // is_active is driven purely from the config object, per assignment
30         // requirement - never hardcoded here.
31         this.is_active = cfg.is_active;
32
33         monitor = router_monitor::type_id::create("monitor", this);
34
35         if (is_active == UVM_ACTIVE) begin
36             driver = router_driver::type_id::create("driver", this);
37             sequencer = router_sequencer::type_id::create("sequencer", this);
38         end
39     endfunction
40
41     function void connect_phase(uvm_phase phase);
42         super.connect_phase(phase);
43         if (is_active == UVM_ACTIVE) begin
44             driver.seq_item_port.connect(sequencer.seq_item_export);
45         end
46     endfunction
47
48 endclass : router_agent

```

Fig 2: UVM Agent class defining component instantiation (driver, monitor, sequencer) based on dynamic active/passive configuration settings.



```
1  //-----
2  // router_config.sv
3  // Environment-wide configuration object, set from the test via
4  // uvm_config_db and retrieved by each component in its build_phase.
5  //-----
6  class router_config extends uvm_object;
7      `uvm_object_utils(router_config)
8
9      // Agent behaviour
10     uvm_active_passive_enum is_active = UVM_ACTIVE;
11
12     // DUT topology (fixed by router.v: 4 input ports, 2 output ports).
13     // Kept in the config object so the environment / coverage never
14     // hardcode the numbers and so the topology is documented in one place.
15     int unsigned num_in_ports  = 4;
16     int unsigned num_out_ports = 2;
17
18     // Feature enables
19     bit coverage_enable = 1;
20     bit scoreboard_enable = 1;
21
22     // Virtual interface handle - set once at the top level (tb_top) and
23     // fetched by driver / monitor / coverage / scoreboard.
24     virtual router_if vif;
25
26     function new(string name = "router_config");
27         super.new(name);
28     endfunction
29
30 endclass : router_config
```

Fig 3: Environment configuration object storing active/passive mode, DUT port sizes, feature enables, and virtual interface handles.

```

router_coverage.sv -- part 1/2

1 //-----
2 // router_coverage.sv
3 // Standalone coverage collector. Per the assignment it needs no TLM
4 // connections: it grabs the same virtual interface handle from the
5 // config object and samples its own covergroups directly off the bus
6 // every clock cycle.
7 //-----
8 class router_coverage extends uvm_component;
9     `uvm_component_utils(router_coverage)
10
11     virtual router_if vif;
12     router_config    cfg;
13
14     bit [7:0] data_in0_s, data_in1_s, data_in2_s, data_in3_s;
15     bit      valid_in0_s, valid_in1_s, valid_in2_s, valid_in3_s;
16     bit      valid_out0_s, valid_out1_s;
17
18     // Covers whether each input channel was seen valid/invalid, and
19     // whether each output channel fired, plus the arbitration cases
20     // (both contending inputs valid at once) on each output.
21     covergroup cg_router;
22         option.per_instance = 1;
23
24         cp_valid_in0 : coverpoint valid_in0_s;
25         cp_valid_in1 : coverpoint valid_in1_s;
26         cp_valid_in2 : coverpoint valid_in2_s;
27         cp_valid_in3 : coverpoint valid_in3_s;
28
29         cp_valid_out0 : coverpoint valid_out0_s;
30         cp_valid_out1 : coverpoint valid_out1_s;
31
32         // Arbitration: out0 is fed by in0 (priority) and in2; out1 by in1
33         // (priority) and in3. Cross to make sure both contention cases and
34         // both idle cases have been exercised on each output.
35         cx_out0_arb : cross cp_valid_in0, cp_valid_in2;
36         cx_out1_arb : cross cp_valid_in1, cp_valid_in3;
37     endgroup
38
39     function new(string name, uvm_component parent);
40         super.new(name, parent);
41         cg_router = new();
42     endfunction
43
44     function void build_phase(uvm_phase phase);
45         super.build_phase(phase);
46         `uvm_info(get_type_name(), "[router_coverage] build_phase @ UVM_LOW", UVM_LOW)
47         `uvm_info(get_type_name(), "[router_coverage] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
48         `uvm_info(get_type_name(), "[router_coverage] build_phase @ UVM_HIGH", UVM_HIGH)
49         `uvm_info(get_type_name(), "[router_coverage] build_phase @ UVM_FULL", UVM_FULL)
50
51         if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
52             `uvm_fatal(get_type_name(), "router_config not found in config_db")
53         end
54         vif = cfg.vif;
55     endfunction
56
57     task run_phase(uvm_phase phase);
58         if (!cfg.coverage_enable) return;
59
60         wait (vif.rst_n == 1'b1);

```

Fig 4: Functional coverage collector component declaring coverpoints and cross-coverage for input/output valid signals and arbitration logic (part 1/2).

```

router_coverage.sv -- part 2/2

61     forever begin
62         @(posedge vif.clk);
63         #1;
64         valid_in0_s = vif.valid_in0; valid_in1_s = vif.valid_in1;
65         valid_in2_s = vif.valid_in2; valid_in3_s = vif.valid_in3;
66         valid_out0_s = vif.valid_out0; valid_out1_s = vif.valid_out1;
67         cg_router.sample();
68         `uvm_info(get_type_name(),
69             $sformatf("Sampled coverage, current cg_router=%0.2f%%", cg_router.get_coverage()),
70             UVM_FULL)
71     end
72 endtask
73
74 function void report_phase(uvm_phase phase);
75     `uvm_info(get_type_name(),
76         $sformatf("Final functional coverage = %0.2f%%", cg_router.get_coverage()), UVM_LOW)
77 endfunction
78
79 endclass : router_coverage

```

Fig 5: Sampling loop implementation for functional coverage triggered on clock edges, featuring full reporting at execution end (part 2/2).

```

router_driver.sv -- part 1/2

1  //-----
2  // router_driver.sv
3  // Drives router_seq_item transactions onto the DUT interface.
4  //
5  // Protocol timing: router.v is purely combinational (data_out reacts in
6  // the same delta cycle as data_in/valid_in change), so the driver
7  // synchronizes stimulus changes to the testbench clock edge; it applies
8  // new inputs shortly after posedge clk and holds them until the next
9  // posedge, giving the monitor a full, stable clock period in which to
10 // sample the combinational response.
11 //-----
12 class router_driver extends uvm_driver #(router_seq_item);
13     `uvm_component_utils(router_driver)
14
15     virtual router_if vif;
16     router_config cfg;
17
18     function new(string name, uvm_component parent);
19         super.new(name, parent);
20     endfunction
21
22     function void build_phase(uvm_phase phase);
23         super.build_phase(phase);
24         `uvm_info(get_type_name(), "[router_driver] build_phase @ UVM_LOW", UVM_LOW)
25         `uvm_info(get_type_name(), "[router_driver] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
26         `uvm_info(get_type_name(), "[router_driver] build_phase @ UVM_HIGH", UVM_HIGH)
27         `uvm_info(get_type_name(), "[router_driver] build_phase @ UVM_FULL", UVM_FULL)
28
29         if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
30             `uvm_fatal(get_type_name(), "router_config not found in config_db")
31         end
32         vif = cfg.vif;
33         if (vif == null) begin
34             `uvm_fatal(get_type_name(), "Virtual interface handle inside router_config is null")
35         end
36     endfunction
37
38     task run_phase(uvm_phase phase);
39         // Hold interface in idle until reset deasserts
40         drive_idle();
41         wait (vif.rst_n == 1'b1);
42         @(posedge vif.clk);
43
44         forever begin
45             router_seq_item req;
46             seq_item_port.get_next_item(req);
47             drive_transaction(req);
48             seq_item_port.item_done();
49         end
50     endtask
51
52     task drive_idle();
53         vif.data_in0 <= 8'h00; vif.data_in1 <= 8'h00;
54         vif.data_in2 <= 8'h00; vif.data_in3 <= 8'h00;
55         vif.valid_in0 <= 1'b0; vif.valid_in1 <= 1'b0;
56         vif.valid_in2 <= 1'b0; vif.valid_in3 <= 1'b0;
57     endtask
58
59     task drive_transaction(router_seq_item req);
60         @(posedge vif.clk);

```

Fig 6: Driver component responsible for retrieving sequence items and driving stimulus signals onto the virtual interface (part 1/2).

```

router_driver.sv — part 2/2

61     vif.data_in0 <= req.data_in[0];
62     vif.data_in1 <= req.data_in[1];
63     vif.data_in2 <= req.data_in[2];
64     vif.data_in3 <= req.data_in[3];
65     vif.valid_in0 <= req.valid_in[0];
66     vif.valid_in1 <= req.valid_in[1];
67     vif.valid_in2 <= req.valid_in[2];
68     vif.valid_in3 <= req.valid_in[3];
69     `uvm_info(get_type_name(),
70               $sformatf("Driving transaction: %s", req.convert2string()), UVM_HIGH)
71   endtask
72
73   endclass : router_driver

```

Fig 7: Helper task implementation assigning sequence item payload fields directly to interface signal arrays (part 2/2).

```

router_env.sv

1  //-----
2  // router_env.sv
3  // Top-level environment instantiating the agent, coverage collector and
4  // scoreboard. Passes the router_config down to the agent via config_db
5  // (the agent then derives its own is_active from that object).
6  //-----
7  class router_env extends uvm_env;
8      `uvm_component_utils(router_env)
9
10     router_agent    agent;
11     router_coverage  coverage;
12     router_scoreboard scoreboard;
13     router_config    cfg;
14
15     function new(string name, uvm_component parent);
16         super.new(name, parent);
17     endfunction
18
19     function void build_phase(uvm_phase phase);
20         super.build_phase(phase);
21         `uvm_info(get_type_name(), "[router_env] build_phase @ UVM_LOW", UVM_LOW)
22         `uvm_info(get_type_name(), "[router_env] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
23         `uvm_info(get_type_name(), "[router_env] build_phase @ UVM_HIGH", UVM_HIGH)
24         `uvm_info(get_type_name(), "[router_env] build_phase @ UVM_FULL", UVM_FULL)
25
26         if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
27             `uvm_fatal(get_type_name(), "router_config not found in config_db")
28         end
29
30         agent      = router_agent::type_id::create("agent", this);
31         coverage    = router_coverage::type_id::create("coverage", this);
32         scoreboard  = router_scoreboard::type_id::create("scoreboard", this);
33     endfunction
34
35     // No connect_phase needed: coverage and scoreboard sample the
36     // interface independently (see their own build_phase), satisfying the
37     // assignment's "No connections needed" requirement. No manual re-publish
38     // of cfg into child contexts is needed either: it was set with a ""
39     // wildcard scope from the test, so every component in the hierarchy
40     // (agent, driver, monitor, sequencer, coverage, scoreboard) can fetch
41     // it directly via uvm_config_db#(router_config)::get(this, "", "cfg", cfg).
42
43   endclass : router_env

```

Fig 8: Top-level UVM environment encapsulating agent, standalone coverage collector, scoreboard, and shared configuration settings.

```

1 //-----
2 // router_if.sv
3 // Interface wrapping the router.v DUT ports.
4 //-----
5 interface router_if (input logic clk, input logic rst_n);
6
7     logic [7:0] data_in0, data_in1, data_in2, data_in3;
8     logic       valid_in0, valid_in1, valid_in2, valid_in3;
9     logic [7:0] data_out0, data_out1;
10    logic       valid_out0, valid_out1;
11
12    // Modport used by the DUT instantiation in tb_top
13    modport DUT (
14        input  clk, rst_n,
15        input  data_in0, data_in1, data_in2, data_in3,
16        input  valid_in0, valid_in1, valid_in2, valid_in3,
17        output data_out0, data_out1, valid_out0, valid_out1
18    );
19
20    endinterface : router_if

```

Fig 9: SystemVerilog interface definition bundling clock, reset, multi-channel data, and valid control signals alongside DUT modport declarations.

```

1 //-----
2 // router_monitor.sv
3 // Samples the router interface every clock cycle (once inputs have
4 // settled and the combinational outputs have propagated) and broadcasts
5 // the observed transaction on its analysis port. Works whether the
6 // agent is UVM_ACTIVE or UVM_PASSIVE.
7 //-----
8 class router_monitor extends uvm_monitor;
9     `uvm_component_utils(router_monitor)
10
11     virtual router_if vif;
12     router_config  cfg;
13
14     uvm_analysis_port #(router_seq_item) ap;
15
16     function new(string name, uvm_component parent);
17         super.new(name, parent);
18         ap = new("ap", this);
19     endfunction
20
21     function void build_phase(uvm_phase phase);
22         super.build_phase(phase);
23         `uvm_info(get_type_name(), "[router_monitor] build_phase @ UVM_LOW", UVM_LOW)
24         `uvm_info(get_type_name(), "[router_monitor] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
25         `uvm_info(get_type_name(), "[router_monitor] build_phase @ UVM_HIGH", UVM_HIGH)
26         `uvm_info(get_type_name(), "[router_monitor] build_phase @ UVM_FULL", UVM_FULL)
27
28         if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
29             `uvm_fatal(get_type_name(), "router_config not found in config_db")
30         end
31         vif = cfg.vif;
32         if (vif == null) begin
33             `uvm_fatal(get_type_name(), "Virtual interface handle inside router_config is null")
34         end
35     endfunction
36
37     task run_phase(uvm_phase phase);
38         wait (vif.rst_n == 1'b1);
39         forever begin
40             router_seq_item txn;
41             @(posedge vif.clk);
42             #1; // let the combinational outputs settle before sampling
43
44             txn = router_seq_item::type_id::create("txn");
45             txn.data_in[0] = vif.data_in0; txn.data_in[1] = vif.data_in1;
46             txn.data_in[2] = vif.data_in2; txn.data_in[3] = vif.data_in3;
47             txn.valid_in[0] = vif.valid_in0; txn.valid_in[1] = vif.valid_in1;
48             txn.valid_in[2] = vif.valid_in2; txn.valid_in[3] = vif.valid_in3;
49             txn.data_out[0] = vif.data_out0; txn.data_out[1] = vif.data_out1;
50             txn.valid_out[0] = vif.valid_out0; txn.valid_out[1] = vif.valid_out1;
51
52             `uvm_info(get_type_name(),
53                 $format("Observed transaction: %s", txn.convert2string()), UVM_HIGH)
54
55             ap.write(txn);
56         end
57     endtask
58
59 endclass : router_monitor

```

Fig 10: UVM monitor component continuously sampling interface signals post-edge and broadcasting transaction objects over an analysis port.



```
1 //-----
2 // router_pkg.sv
3 // Package that pulls together every UVM class in the correct
4 // dependency order. router_if.sv is NOT included here because
5 // interfaces cannot live inside a package - it is compiled/imported
6 // separately (see tb_top.sv).
7 //-----
8 package router_pkg;
9
10     import uvm_pkg::*;
11     `include "uvm_macros.svh"
12
13     `include "router_config.sv"
14     `include "router_seq_item.sv"
15     `include "router_sequence.sv"
16     `include "router_sequencer.sv"
17     `include "router_driver.sv"
18     `include "router_monitor.sv"
19     `include "router_agent.sv"
20     `include "router_coverage.sv"
21     `include "router_scoreboard.sv"
22     `include "router_env.sv"
23     `include "router_test.sv"
24
25 endpackage : router_pkg
```

Fig 11: Package file ordering macro imports and file inclusion headers to compile the entire UVM testbench hierarchy in proper dependency order.

```

router_scoreboard.sv — part 1/2

1 //-----
2 // router_scoreboard.sv
3 // Self-checking scoreboard. Per the assignment it needs no TLM
4 // connections: it independently samples the same virtual interface,
5 // computes the expected routing result from the reference model
6 // (mirroring router.v's documented behaviour: even inputs -> out0 with
7 // in0 priority over in2, odd inputs -> out1 with in1 priority over in3)
8 // and compares it against what the DUT actually produced that cycle.
9 //-----
10 class router_scoreboard extends uvm_component;
11     `uvm_component_utils(router_scoreboard)
12
13     virtual router_if vif;
14     router_config  cfg;
15
16     int unsigned num_checked = 0;
17     int unsigned num_errors  = 0;
18
19     function new(string name, uvm_component parent);
20         super.new(name, parent);
21     endfunction
22
23     function void build_phase(uvm_phase phase);
24         super.build_phase(phase);
25         `uvm_info(get_type_name(), "[router_scoreboard] build_phase @ UVM_LOW", UVM_LOW)
26         `uvm_info(get_type_name(), "[router_scoreboard] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
27         `uvm_info(get_type_name(), "[router_scoreboard] build_phase @ UVM_HIGH", UVM_HIGH)
28         `uvm_info(get_type_name(), "[router_scoreboard] build_phase @ UVM_FULL", UVM_FULL)
29
30         if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
31             `uvm_fatal(get_type_name(), "router_config not found in config_db")
32         end
33         vif = cfg.vif;
34     endfunction
35
36     task run_phase(uvm_phase phase);
37         bit [7:0] exp_data_out0, exp_data_out1;
38         bit      exp_valid_out0, exp_valid_out1;
39
40         if (!cfg.scoreboard_enable) return;
41
42         wait (vif.rst_n == 1'b1);
43         forever begin
44             @(posedge vif.clk);
45             #1; // sample after the combinational outputs have settled
46
47             // Reference model
48             exp_data_out0 = vif.valid_in0 ? vif.data_in0 : vif.data_in2;
49             exp_valid_out0 = vif.valid_in0 | vif.valid_in2;
50             exp_data_out1 = vif.valid_in1 ? vif.data_in1 : vif.data_in3;
51             exp_valid_out1 = vif.valid_in1 | vif.valid_in3;
52
53             num_checked++;
54
55             if (exp_valid_out0 != vif.valid_out0 ||
56                 (exp_valid_out0 && exp_data_out0 != vif.data_out0)) begin
57                 num_errors++;
58                 `uvm_error(get_type_name(),
59                     $sprintf("OUT0 MISMATCH: exp data=%0d valid=%0b, got data=%0d valid=%0b",
60                         exp_data_out0, exp_valid_out0, vif.data_out0, vif.valid_out0))
61             end
62         end
63     endtask
64
65     function void report_phase(uvm_phase phase);
66         `uvm_info(get_type_name(),
67             $sprintf("Scoreboard summary: %0d cycles checked, %0d errors", num_checked, num_errors),
68             UVM_LOW)
69     endfunction
70 endclass : router_scoreboard

```

Fig 12: Self-checking UVM scoreboard implementation calculating expected routing logic and checking output

```

router_scoreboard.sv — part 2/2

61     end
62
63     if (exp_valid_out1 != vif.valid_out1 ||
64         (exp_valid_out1 && exp_data_out1 != vif.data_out1)) begin
65         num_errors++;
66         `uvm_error(get_type_name(),
67             $sprintf("OUT1 MISMATCH: exp data=%0d valid=%0b, got data=%0d valid=%0b",
68                 exp_data_out1, exp_valid_out1, vif.data_out1, vif.valid_out1))
69     end
70 endtask
71
72 function void report_phase(uvm_phase phase);
73     `uvm_info(get_type_name(),
74         $sprintf("Scoreboard summary: %0d cycles checked, %0d errors", num_checked, num_errors),
75         UVM_LOW)
76 endfunction
77
78 endclass : router_scoreboard

```

Fig 13: Scoreboard execution logic checking output channel one for mismatches and printing total error summaries during the report phase (part 2/2).

```

router_seq_item.sv — part 1/2

1  //-----
2  // router_seq_item.sv
3  // Transaction modeling one router stimulus cycle: 4 input channels in,
4  // 2 output channels captured back by the monitor after the DUT reacts.
5  //-----
6  class router_seq_item extends uvm_sequence_item;
7
8      // Stimulus (driven by the driver)
9      rand bit [7:0] data_in  [4];
10     rand bit      valid_in  [4];
11
12     // Response (filled in by the monitor when it samples the DUT outputs;
13     // the driver leaves these at their default value)
14     bit [7:0] data_out  [2];
15     bit      valid_out [2];
16
17     `uvm_object_utils(router_seq_item)
18
19     // Bias: make sure valid_in is asserted most of the time so the router
20     // actually has meaningful traffic to route, but still let all-zero
21     // (idle) cycles occur occasionally.
22     constraint c_valid_dist {
23         foreach (valid_in[i]) {
24             valid_in[i] dist {1'b1 := 80, 1'b0 := 20};
25         }
26     }
27
28     function new(string name = "router_seq_item");
29         super.new(name);
30     endfunction
31
32     function void do_copy(uvm_object rhs);
33         router_seq_item rhs_;
34         if (!$cast(rhs_, rhs)) begin
35             `uvm_fatal("DO_COPY", "Cast of rhs object failed")
36         end
37         super.do_copy(rhs);
38         foreach (data_in[i]) data_in[i] = rhs_.data_in[i];
39         foreach (valid_in[i]) valid_in[i] = rhs_.valid_in[i];
40         foreach (data_out[i]) data_out[i] = rhs_.data_out[i];
41         foreach (valid_out[i]) valid_out[i] = rhs_.valid_out[i];
42     endfunction
43
44     function bit do_compare(uvm_object rhs, uvm_comparer comparer);
45         router_seq_item rhs_;
46         bit match;
47         if (!$cast(rhs_, rhs)) return 0;
48         match = 1;
49         foreach (data_in[i]) match &= (data_in[i] == rhs_.data_in[i]);
50         foreach (valid_in[i]) match &= (valid_in[i] == rhs_.valid_in[i]);
51         foreach (data_out[i]) match &= (data_out[i] == rhs_.data_out[i]);
52         foreach (valid_out[i]) match &= (valid_out[i] == rhs_.valid_out[i]);
53         return match;
54     endfunction
55
56     function string convert2string();
57         string s;
58         s = $sprintf(
59             "in0=%0d(v=%0b) in1=%0d(v=%0b) in2=%0d(v=%0b) in3=%0d(v=%0b) || out0=%0d(v=%0b) out1=%0d(v=%0b)",
60             data_in[0], valid_in[0], data_in[1], valid_in[1],

```

Fig 14: UVM sequence item defining randomized input stimulus, expected output data fields, distribution constraints, and copy/compare methods (part 1/2).

```

router_seq_item.sv — part 2/2

61     data_in[2], valid_in[2], data_in[3], valid_in[3],
62     data_out[0], valid_out[0], data_out[1], valid_out[1]);
63     return s;
64 endfunction
65
66 function void do_print(uvm_printer printer);
67     printer.print_string("txn", convert2string());
68 endfunction
69
70 endclass : router_seq_item

```

Fig 15: Sequence item string conversion and custom print utilities for formatting transaction data into readable logs (part 2/2).

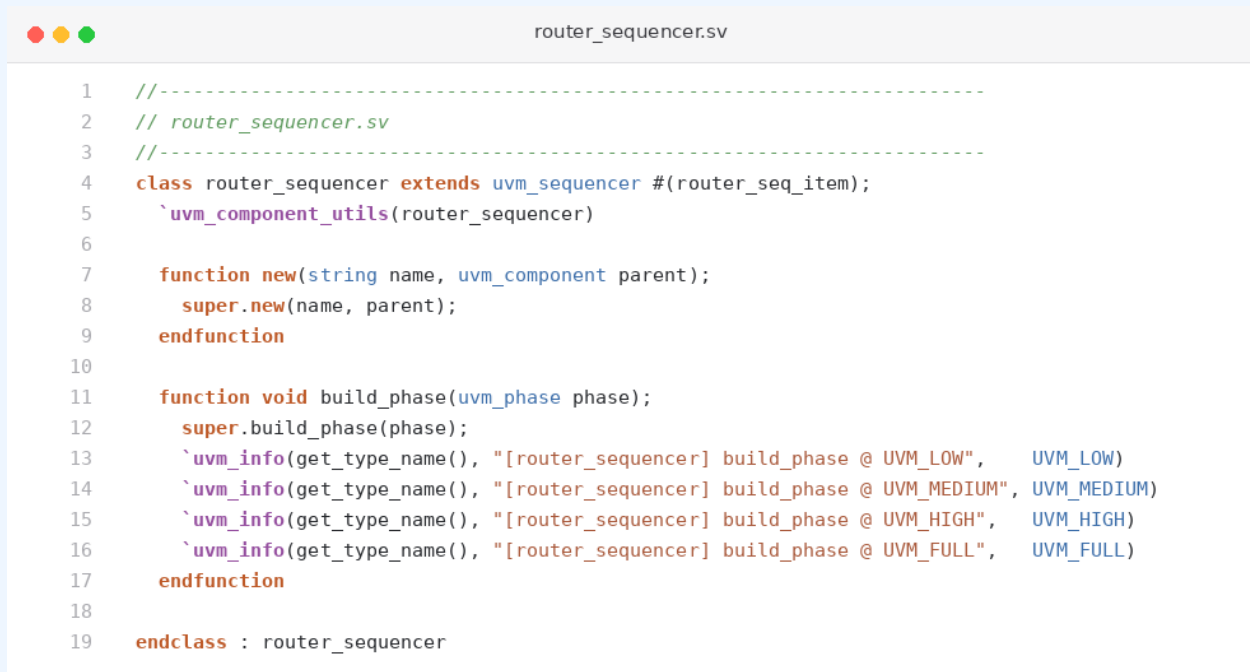
```

router_sequence.sv

1 //-----
2 // router_sequence.sv
3 // Representative sequence: generates a stream of randomized router
4 // transactions, including a couple of directed corner cases
5 // (all-idle cycle, and both channels of a given output contending
6 // at once) layered on top of random traffic.
7 //-----
8 class router_base_sequence extends uvm_sequence #(router_seq_item);
9     `uvm_object_utils(router_base_sequence)
10
11     rand int unsigned num_txns = 20;
12
13     function new(string name = "router_base_sequence");
14         super.new(name);
15     endfunction
16
17     task body();
18         router_seq_item item;
19
20         `uvm_info(get_type_name(),
21             $sformatf("Starting router_base_sequence with %0d transactions", num_txns), UVM_MEDIUM)
22
23         // Directed corner case #1: everything idle
24         item = router_seq_item::type_id::create("idle_item");
25         start_item(item);
26         if (!item.randomize() with {
27             foreach (valid_in[i]) valid_in[i] == 0;
28         }) `uvm_error(get_type_name(), "Randomization failed (idle case)")
29         finish_item(item);
30
31         // Directed corner case #2: contention on both outputs
32         item = router_seq_item::type_id::create("contend_item");
33         start_item(item);
34         if (!item.randomize() with {
35             foreach (valid_in[i]) valid_in[i] == 1;
36         }) `uvm_error(get_type_name(), "Randomization failed (contention case)")
37         finish_item(item);
38
39         // Random traffic
40         repeat (num_txns) begin
41             item = router_seq_item::type_id::create("item");
42             start_item(item);
43             if (!item.randomize())
44                 `uvm_error(get_type_name(), "Randomization failed")
45             finish_item(item);
46         end
47     endtask
48
49 endclass : router_base_sequence

```

Fig 16: Base UVM sequence generating both directed corner-case transactions (idle and contention) alongside randomized payload streams.



```
1  //-----
2  // router_sequencer.sv
3  //-----
4  class router_sequencer extends uvm_sequencer #(router_seq_item);
5      `uvm_component_utils(router_sequencer)
6
7      function new(string name, uvm_component parent);
8          super.new(name, parent);
9      endfunction
10
11     function void build_phase(uvm_phase phase);
12         super.build_phase(phase);
13         `uvm_info(get_type_name(), "[router_sequencer] build_phase @ UVM_LOW", UVM_LOW)
14         `uvm_info(get_type_name(), "[router_sequencer] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
15         `uvm_info(get_type_name(), "[router_sequencer] build_phase @ UVM_HIGH", UVM_HIGH)
16         `uvm_info(get_type_name(), "[router_sequencer] build_phase @ UVM_FULL", UVM_FULL)
17     endfunction
18
19 endclass : router_sequencer
```

Fig 17: UVM sequencer declaration extending `uvm_sequencer` to manage transaction flow between sequences and the driver.

```

router_test.sv -- part 1/2

1 //-----
2 // router_test.sv
3 // Base test: builds the router_config (is_active = UVM_ACTIVE by
4 // default), publishes it through uvm_config_db, instantiates the env,
5 // and runs router_base_sequence on the agent's sequencer.
6 //-----
7 class router_base_test extends uvm_test;
8     `uvm_component_utils(router_base_test)
9
10     router_env  env;
11     router_config cfg;
12
13     function new(string name = "router_base_test", uvm_component parent = null);
14         super.new(name, parent);
15     endfunction
16
17     function void build_phase(uvm_phase phase);
18         super.build_phase(phase);
19         `uvm_info(get_type_name(), "[router_base_test] build_phase @ UVM_LOW", UVM_LOW)
20         `uvm_info(get_type_name(), "[router_base_test] build_phase @ UVM_MEDIUM", UVM_MEDIUM)
21         `uvm_info(get_type_name(), "[router_base_test] build_phase @ UVM_HIGH", UVM_HIGH)
22         `uvm_info(get_type_name(), "[router_base_test] build_phase @ UVM_FULL", UVM_FULL)
23
24         cfg = router_config::type_id::create("cfg");
25         cfg.is_active = UVM_ACTIVE; // switch to UVM_PASSIVE to demo passive mode
26         cfg.num_in_ports = 4;
27         cfg.num_out_ports = 2;
28         cfg.coverage_enable = 1;
29         cfg.scoreboard_enable = 1;
30
31         if (!uvm_config_db#(virtual router_if)::get(this, "", "vif", cfg.vif)) begin
32             `uvm_fatal(get_type_name(), "virtual interface not found in config_db (set by tb_top)")
33         end
34
35         uvm_config_db#(router_config)::set(this, "", "cfg", cfg);
36
37         env = router_env::type_id::create("env", this);
38     endfunction
39
40     task run_phase(uvm_phase phase);
41         router_base_sequence seq;
42         phase.raise_objection(this, "Starting router_base_sequence");
43
44         seq = router_base_sequence::type_id::create("seq");
45         if (!seq.randomize() with { num_txns == 30; })
46             `uvm_warning(get_type_name(), "Sequence randomize failed, using default num_txns")
47
48         seq.start(env.agent.sequencer);
49
50         // Let the last transaction propagate through the DUT and be sampled
51         // by the monitor/coverage/scoreboard before dropping the objection.
52         #20;
53
54         phase.drop_objection(this, "router_base_sequence complete");
55     endtask
56
57 endclass : router_base_test
58
59 //-----
60 // router_passive_test.sv

```

Fig 18: Base UVM test class configuring the environment, storing virtual interface handles, and starting the base sequence with objection handling (part 1/2).



```
61 // Demonstrates UVM_PASSIVE mode: the agent is config'd passive so only
62 // the monitor is built (no driver/sequencer). Useful to show that
63 // is_active truly comes from the config object rather than being
64 // hardcoded in the agent.
65 //-----
66 class router_passive_test extends router_base_test;
67     `uvm_component_utils(router_passive_test)
68
69     function new(string name = "router_passive_test", uvm_component parent = null);
70         super.new(name, parent);
71     endfunction
72
73     function void build_phase(uvm_phase phase);
74         super.build_phase(phase);
75         cfg.is_active = UVM_PASSIVE;
76     endfunction
77
78     // Passive mode has no driver/sequencer to drive stimulus, so this test
79     // simply lets the DUT idle and observes; it's meant to be exercised
80     // together with an external stimulus source or a waveform replay, and
81     // exists here to demonstrate the config-driven active/passive switch.
82     task run_phase(uvm_phase phase);
83         phase.raise_objection(this, "router_passive_test observing");
84         #200;
85         phase.drop_objection(this, "router_passive_test done");
86     endtask
87
88 endclass : router_passive_test
```

Fig 19: Passive mode test class overriding configuration to UVM_PASSIVE for verifying testbench behavior without driver instantiation (part 2/2).

```

1 //-----
2 // tb_top.sv
3 // Top-level testbench: generates clk/rst_n, instantiates the DUT and the
4 // router_if, publishes the virtual interface through uvm_config_db, and
5 // kicks off UVM via run_test().
6 //-----
7 `timescale 1ns/1ps
8
9 import uvm_pkg::*;
10 `include "uvm_macros.svh"
11 import router_pkg::*;
12
13 module tb_top;
14
15     logic clk;
16     logic rst_n;
17
18     // Clock: 10ns period
19     initial clk = 0;
20     always #5 clk = ~clk;
21
22     // Active-low reset, released after a couple of clock edges
23     initial begin
24         rst_n = 0;
25         repeat (3) @(posedge clk);
26         rst_n = 1;
27     end
28
29     router_if vif (.clk(clk), .rst_n(rst_n));
30
31     router dut (
32         .clk          (clk),
33         .rst_n        (rst_n),
34         .data_in0      (vif.data_in0),
35         .data_in1      (vif.data_in1),
36         .data_in2      (vif.data_in2),
37         .data_in3      (vif.data_in3),
38         .valid_in0     (vif.valid_in0),
39         .valid_in1     (vif.valid_in1),
40         .valid_in2     (vif.valid_in2),
41         .valid_in3     (vif.valid_in3),
42         .data_out0     (vif.data_out0),
43         .data_out1     (vif.data_out1),
44         .valid_out0    (vif.valid_out0),
45         .valid_out1    (vif.valid_out1)
46     );
47
48     initial begin
49         uvm_config_db#(virtual router_if)::set(null, "", "vif", vif);
50         run_test(); // test name supplied on the command line via +UVM_TESTNAME
51     end
52
53     // Safety timeout so a stuck simulation doesn't run forever
54     initial begin
55         #100000;
56         `uvm_fatal("TB_TOP", "Simulation timeout - watchdog expired")
57     end
58
59 endmodule : tb_top

```

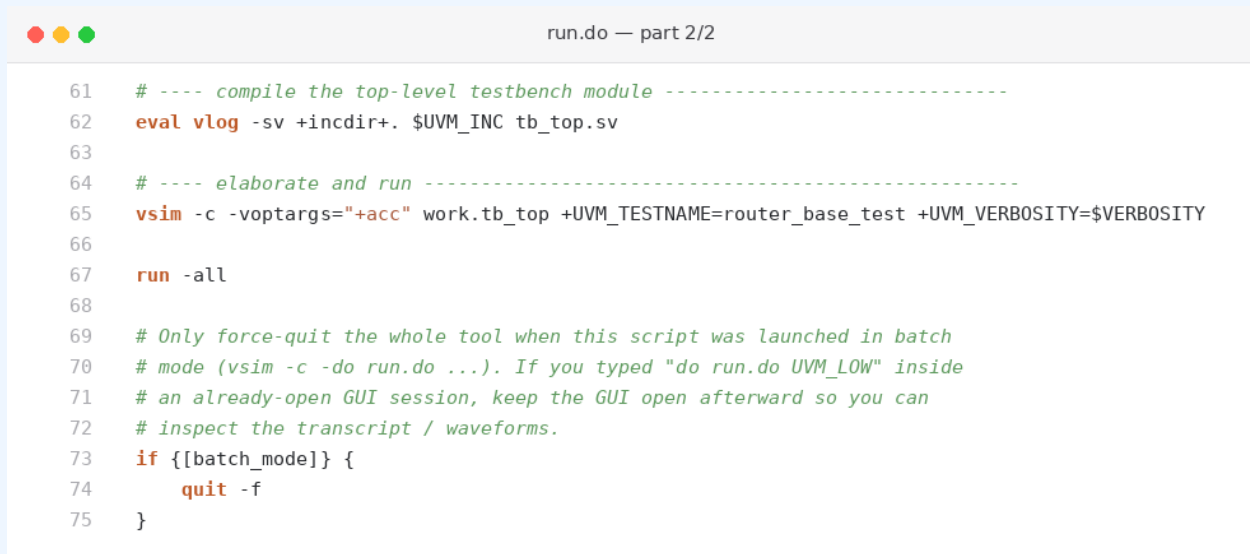
Fig 20: Top-level SystemVerilog testbench module instantiating the DUT and interface, generating clock/reset signals, publishing the virtual interface handle to uvm_config_db, and invoking run_test().

SNIPPET OF THE .DO FILE

```
run.do — part 1/2

1  #-----
2  # run.do - Assignment 5 automation script (ModelSim / Questa)
3  #
4  # Compiles the DUT + full UVM testbench once, then elaborates and runs.
5  # The ONLY thing that changes between the four required runs is the
6  # +UVM_VERBOSEITY plusarg passed as an argument to this script - no
7  # source file is ever edited between runs.
8  #
9  # Usage (from the ModelSim/Questa "do" prompt, an OS shell, or a batch
10 # script), run once per verbosity level:
11 #
12 # vsim -c -do "do run.do UVM_LOW"
13 # vsim -c -do "do run.do UVM_MEDIUM"
14 # vsim -c -do "do run.do UVM_HIGH"
15 # vsim -c -do "do run.do UVM_FULL"
16 #
17 # Or, equivalently, from an OS shell in batch mode:
18 # vsim -c -do "do run.do UVM_LOW; quit -f"
19 #-----
20
21 # ---- resolve requested verbosity (defaults to UVM_MEDIUM if omitted) --
22 if {[info exists 1]} {
23     set VERBOSITY $1
24 } else {
25     set VERBOSITY "UVM_MEDIUM"
26 }
27 echo "=== run.do: requested verbosity = $VERBOSITY ==="
28
29 # ---- clean + (re)build the work library -----
30 if {[file exists work]} {
31     vdel -lib work -all
32 }
33 vlib work
34 vmap work work
35
36 # ---- compile DUT -----
37 vlog -sv +incdir+. router.v
38
39 # ---- compile interface (must precede the package: classes inside the
40 # package reference the global "router_if" type) -----
41 vlog -sv +incdir+. router_if.v
42
43 # ---- locate UVM headers (uvm_macros.svh) -----
44 # Prefer an explicitly-set UVM_HOME. If it isn't set, fall back to no extra
45 # +incdir - most Questa installs already resolve `include "uvm_macros.svh"
46 # against their bundled UVM library without any extra path. If your
47 # compile fails on uvm_macros.svh, set UVM_HOME once before calling this
48 # script, e.g. (Transcript window):
49 # set env(UVM_HOME) {C:/questasim64_10.6c/verilog_src/uvm-1.2}
50 if {[info exists ::env(UVM_HOME)]} {
51     set UVM_INC "+incdir+${::env(UVM_HOME)}/src"
52     echo "=== run.do: using UVM_HOME=${::env(UVM_HOME)} ==="
53 } else {
54     set UVM_INC ""
55     echo "=== run.do: UVM_HOME not set, compiling without extra +incdir ==="
56 }
57
58 # ---- compile the UVM package (pulls in every class via `include) -----
59 eval vlog -sv +incdir+. $UVM_INC router_pkg.v
60
```

Fig 21: ModelSim/Questa simulation DO script handling verbosity parameters, work library creation, and compilation of SystemVerilog design files (part 1/2).



```
61  # ---- compile the top-level testbench module ----  
62  eval vlog -sv +incdir+. $UVM_INC tb_top.sv  
63  
64  # ---- elaborate and run ----  
65  vsim -c -voptargs="+acc" work.tb_top +UVM_TESTNAME=router_base_test +UVM_VERBOSITY=$VERBOSITY  
66  
67  run -all  
68  
69  # Only force-quit the whole tool when this script was launched in batch  
70  # mode (vsim -c -do run.do ...). If you typed "do run.do UVM_LOW" inside  
71  # an already-open GUI session, keep the GUI open afterward so you can  
72  # inspect the transcript / waveforms.  
73  if {[batch_mode]} {  
74      quit -f  
75  }
```

Fig 22: Simulation script commands compiling the testbench top module, launching execution with target parameters, and managing batch termination (part 2/2).

LOG SNIPPETS FOR ALL FOUR RUNS

```
UVM_INFO @ 0: reporter [RNTST] Running test router_base_test...
UVM_INFO router_test.sv(19) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_LOW
UVM_INFO router_env.sv(21) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_LOW
UVM_INFO router_agent.sv(20) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_LOW
UVM_INFO router_driver.sv(24) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_LOW
UVM_INFO router_monitor.sv(23) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_LOW
UVM_INFO router_sequencer.sv(13) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_LOW
UVM_INFO router_coverage.sv(46) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_LOW
UVM_INFO router_scoreboard.sv(25) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_LOW
UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.sv(1268) @ 355020: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO router_coverage.sv(76) @ 355020: uvm_test_top.env.coverage [router_coverage] Final functional coverage = 100.00%
UVM_INFO router_scoreboard.sv(76) @ 355020: uvm_test_top.env.scoreboard [router_scoreboard] Scoreboard summary: 33 cycles checked, 0 errors

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 12
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0

** Report counts by id
[RNTST] 1
[TEST_DONE] 1
[router_agent] 1
[router_base_test] 1
[router_coverage] 2
[router_driver] 1
[router_env] 1
[router_monitor] 1
[router_scoreboard] 2
[router_sequencer] 1

** Note: $finish : C:/questasim64_10.6c/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.sv(430)
Time: 355020 ps Iteration: 54 Instance: /tb_top
```

Fig 23: Transcript log operating under UVM_LOW verbosity setting, filtering out transaction-level messaging to display only primary phase messages and the final report summary.

```
UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(215) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(217) @ 0: reporter [Questa UVM] questa_uvm::init(+struct)
UVM_INFO @ 0: reporter [RNTST] Running test router_base_test...
UVM_INFO router_test.sv(19) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_LOW
UVM_INFO router_test.sv(20) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_MEDIUM
UVM_INFO router_env.sv(21) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_LOW
UVM_INFO router_env.sv(22) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_MEDIUM
UVM_INFO router_agent.sv(20) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_LOW
UVM_INFO router_agent.sv(21) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_MEDIUM
UVM_INFO router_driver.sv(24) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_LOW
UVM_INFO router_driver.sv(25) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_MEDIUM
UVM_INFO router_monitor.sv(23) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_LOW
UVM_INFO router_monitor.sv(24) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_MEDIUM
UVM_INFO router_sequencer.sv(13) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_LOW
UVM_INFO router_sequencer.sv(14) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_MEDIUM
UVM_INFO router_coverage.sv(46) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_LOW
UVM_INFO router_coverage.sv(47) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_MEDIUM
UVM_INFO router_scoreboard.sv(25) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_LOW
UVM_INFO router_scoreboard.sv(26) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_MEDIUM
UVM_INFO router_sequence.sv(21) @ 0: uvm_test_top.env.agent.sequencer[Seq] [router_base_sequence] Starting router_base_sequence with 30 transactions
UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.sv(1268) @ 355020: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
UVM_INFO router_coverage.sv(76) @ 355020: uvm_test_top.env.coverage [router_coverage] Final functional coverage = 100.00%
UVM_INFO router_scoreboard.sv(76) @ 355020: uvm_test_top.env.scoreboard [router_scoreboard] Scoreboard summary: 33 cycles checked, 0 errors

--- UVM Report Summary ---

** Report counts by severity
UVM_INFO : 23
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0

** Report counts by id
[Questa UVM] 2
[RNTST] 1
[TEST_DONE] 1
[router_agent] 2
[router_base_sequence] 1
[router_base_test] 2
[router_coverage] 3
[router_driver] 2
[router_env] 2
[router_monitor] 2
[router_scoreboard] 3
[router_sequencer] 2

** Note: $finish : C:/questasim64_10.6c/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.sv(430)
Time: 355020 ps Iteration: 54 Instance: /tb_top
```

Fig 24: Transcript log running under UVM_MEDIUM verbosity, providing intermediate message logging alongside testbench lifecycle steps and summary statistics.

```
Transcript
File Edit View Bookmarks Window Help
Transcript
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(215) @ 0: reporter [Questa UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(217) @ 0: reporter [Questa UVM] questa_uvm::init(+struct)
# UVM_INFO @ 0: reporter [RNTST] Running test router_base_test...
# UVM_INFO router_test.sv(19) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_LOW
# UVM_INFO router_test.sv(20) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_MEDIUM
# UVM_INFO router_test.sv(21) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_HIGH
# UVM_INFO router_env.sv(21) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_LOW
# UVM_INFO router_env.sv(22) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_MEDIUM
# UVM_INFO router_env.sv(23) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_HIGH
# UVM_INFO router_monitor.sv(23) @ 0: uvm_test_top.env.agent_monitor [router_monitor] [router_monitor] build_phase @ UVM_LOW
# UVM_INFO router_agent.sv(20) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_LOW
# UVM_INFO router_agent.sv(21) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_MEDIUM
# UVM_INFO router_agent.sv(22) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_HIGH
# UVM_INFO router_driver.sv(24) @ 0: uvm_test_top.env.agent_driver [router_driver] [router_driver] build_phase @ UVM_LOW
# UVM_INFO router_driver.sv(25) @ 0: uvm_test_top.env.agent_driver [router_driver] [router_driver] build_phase @ UVM_MEDIUM
# UVM_INFO router_driver.sv(26) @ 0: uvm_test_top.env.agent_driver [router_driver] [router_driver] build_phase @ UVM_HIGH
# UVM_INFO router_monitor.sv(23) @ 0: uvm_test_top.env.agent_monitor [router_monitor] [router_monitor] build_phase @ UVM_LOW
# UVM_INFO router_monitor.sv(24) @ 0: uvm_test_top.env.agent_monitor [router_monitor] [router_monitor] build_phase @ UVM_MEDIUM
# UVM_INFO router_monitor.sv(25) @ 0: uvm_test_top.env.agent_monitor [router_monitor] [router_monitor] build_phase @ UVM_HIGH
# UVM_INFO router_sequencer.sv(13) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_LOW
# UVM_INFO router_sequencer.sv(14) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_MEDIUM
# UVM_INFO router_sequencer.sv(15) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_HIGH
# UVM_INFO router_coverage.sv(46) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_LOW
# UVM_INFO router_coverage.sv(47) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_MEDIUM
# UVM_INFO router_coverage.sv(48) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_HIGH
# UVM_INFO router_scoreboard.sv(25) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_LOW
# UVM_INFO router_scoreboard.sv(26) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_MEDIUM
# UVM_INFO router_scoreboard.sv(27) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_HIGH
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questa UVM] End of Elaboration
# UVM_INFO router_sequencer.sv(21) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] Starting router_base_sequence with 30 transactions
# UVM_INFO router_driver.sv(70) @ 45000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=69(v=0) in1=144(v=0) in2=24(v=0) in3=132(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 45001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=69(v=0) in1=144(v=0) in2=24(v=0) in3=132(v=0) || out0=24(v=0) out1=132(v=0)
# UVM_INFO router_driver.sv(70) @ 45001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=164(v=1) in1=69(v=1) in2=1(v=1) in3=65(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 45001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=164(v=1) in1=69(v=1) in2=1(v=1) in3=65(v=1) || out0=164(v=1) out1=69(v=1)
# UVM_INFO router_driver.sv(70) @ 45001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=123(v=1) in1=223(v=0) in2=237(v=0) in3=162(v=1) || out0=123(v=1) out1=162(v=1)
# UVM_INFO router_monitor.sv(53) @ 45001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=123(v=1) in1=223(v=0) in2=237(v=0) in3=162(v=1) || out0=123(v=1) out1=162(v=1)
# UVM_INFO router_driver.sv(70) @ 45001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=164(v=0) in1=224(v=1) in2=223(v=1) in3=236(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 45001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=164(v=0) in1=224(v=1) in2=223(v=1) in3=236(v=0) || out0=223(v=1) out1=224(v=1)
# UVM_INFO router_driver.sv(70) @ 45001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=202(v=1) in1=188(v=1) in2=42(v=1) in3=77(v=0) || out0=202(v=1) out1=188(v=1)
# UVM_INFO router_monitor.sv(53) @ 45001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=202(v=1) in1=188(v=1) in2=42(v=1) in3=77(v=0) || out0=202(v=1) out1=188(v=1)
# UVM_INFO router_driver.sv(70) @ 45001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=86(v=0) in1=144(v=1) in2=14(v=0) in3=2(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 45001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=86(v=0) in1=144(v=1) in2=14(v=0) in3=2(v=1) || out0=14(v=0) out1=144(v=1)
# UVM_INFO router_driver.sv(70) @ 105000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=17(v=1) in1=109(v=0) in2=81(v=1) in3=144(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 105001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=17(v=1) in1=109(v=0) in2=81(v=1) in3=144(v=1) || out0=17(v=1) out1=144(v=1)
# UVM_INFO router_driver.sv(70) @ 115000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=177(v=1) in1=194(v=0) in2=230(v=1) in3=10(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 115001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=177(v=1) in1=194(v=0) in2=230(v=1) in3=10(v=1) || out0=177(v=1) out1=10(v=1)
# UVM_INFO router_driver.sv(70) @ 125000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=42(v=0) in1=130(v=1) in2=243(v=1) in3=172(v=0) || out0=0(v=0) out1=0(v=0)
```

Fig 25: Simulation transcript log under UVM_HIGH verbosity displaying component build phase execution and initial sequence transaction logs (part 1/3).

```
Transcript
File Edit View Bookmarks Window Help
Transcript
# UVM_INFO router_monitor.sv(53) @ 125001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=42(v=0) in1=130(v=1) in2=243(v=1) in3=172(v=0) || out0=243(v=1) out1=130(v=1)
# UVM_INFO router_driver.sv(70) @ 125001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=126(v=1) in1=49(v=0) in2=225(v=0) in3=251(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 135001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=126(v=1) in1=49(v=0) in2=225(v=0) in3=251(v=1) || out0=126(v=1) out1=251(v=1)
# UVM_INFO router_driver.sv(70) @ 145000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=103(v=0) in1=138(v=1) in2=99(v=0) in3=61(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 145001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=103(v=0) in1=138(v=1) in2=99(v=0) in3=61(v=1) || out0=99(v=0) out1=138(v=1)
# UVM_INFO router_driver.sv(70) @ 145001: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=161(v=1) in1=140(v=1) in2=197(v=1) in3=228(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 155001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=161(v=1) in1=140(v=1) in2=197(v=1) in3=228(v=1) || out0=161(v=1) out1=140(v=1)
# UVM_INFO router_driver.sv(70) @ 165000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=172(v=1) in1=5(v=1) in2=172(v=1) in3=200(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 165001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=172(v=1) in1=5(v=1) in2=172(v=1) in3=200(v=1) || out0=172(v=1) out1=5(v=1)
# UVM_INFO router_driver.sv(70) @ 175000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=99(v=1) in1=53(v=1) in2=188(v=1) in3=15(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 175001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=99(v=1) in1=53(v=1) in2=188(v=1) in3=15(v=1) || out0=99(v=1) out1=53(v=1)
# UVM_INFO router_driver.sv(70) @ 185000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=89(v=1) in1=170(v=1) in2=175(v=1) in3=155(v=1) || out0=89(v=1) out1=170(v=1)
# UVM_INFO router_monitor.sv(53) @ 185001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=89(v=1) in1=170(v=1) in2=175(v=1) in3=155(v=1) || out0=89(v=1) out1=170(v=1)
# UVM_INFO router_driver.sv(70) @ 195000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=87(v=1) in1=121(v=1) in2=176(v=1) in3=191(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 195001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=87(v=1) in1=121(v=1) in2=176(v=1) in3=191(v=0) || out0=87(v=1) out1=121(v=1)
# UVM_INFO router_driver.sv(70) @ 205000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=124(v=0) in1=223(v=1) in2=148(v=1) in3=80(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 205001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=124(v=0) in1=223(v=1) in2=148(v=1) in3=80(v=1) || out0=148(v=1) out1=223(v=1)
# UVM_INFO router_driver.sv(70) @ 215000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=209(v=0) in1=17(v=1) in2=243(v=1) in3=133(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 215001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=209(v=0) in1=17(v=1) in2=243(v=1) in3=133(v=1) || out0=243(v=1) out1=17(v=1)
# UVM_INFO router_driver.sv(70) @ 225000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=193(v=1) in1=42(v=1) in2=211(v=1) in3=225(v=1) || out0=193(v=1) out1=42(v=1)
# UVM_INFO router_monitor.sv(53) @ 225001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=193(v=1) in1=42(v=1) in2=211(v=1) in3=225(v=1) || out0=193(v=1) out1=42(v=1)
# UVM_INFO router_driver.sv(70) @ 235000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=213(v=1) in1=95(v=1) in2=226(v=1) in3=135(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 235001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=213(v=1) in1=95(v=1) in2=226(v=1) in3=135(v=1) || out0=213(v=1) out1=95(v=1)
# UVM_INFO router_driver.sv(70) @ 245000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=29(v=1) in1=239(v=1) in2=68(v=1) in3=87(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 245001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=29(v=1) in1=239(v=1) in2=68(v=1) in3=87(v=0) || out0=29(v=1) out1=239(v=1)
# UVM_INFO router_driver.sv(70) @ 255000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=194(v=0) in1=130(v=1) in2=248(v=0) in3=113(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 255001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=194(v=0) in1=130(v=1) in2=248(v=0) in3=113(v=1) || out0=248(v=0) out1=130(v=1)
# UVM_INFO router_driver.sv(70) @ 265000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=136(v=1) in1=174(v=1) in2=21(v=1) in3=9(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 265001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=136(v=1) in1=174(v=1) in2=21(v=1) in3=9(v=1) || out0=136(v=1) out1=174(v=1)
# UVM_INFO router_driver.sv(70) @ 275000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=34(v=1) in1=97(v=0) in2=176(v=1) in3=125(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 275001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=34(v=1) in1=97(v=0) in2=176(v=1) in3=125(v=1) || out0=34(v=1) out1=125(v=1)
# UVM_INFO router_driver.sv(70) @ 285000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=175(v=1) in1=209(v=0) in2=128(v=1) in3=74(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 285001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=175(v=1) in1=209(v=0) in2=128(v=1) in3=74(v=1) || out0=175(v=1) out1=74(v=1)
# UVM_INFO router_driver.sv(70) @ 295000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=44(v=1) in1=87(v=1) in2=45(v=1) in3=211(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 295001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=44(v=1) in1=87(v=1) in2=45(v=1) in3=211(v=1) || out0=44(v=1) out1=87(v=1)
# UVM_INFO router_driver.sv(70) @ 305000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=122(v=0) in1=109(v=1) in2=123(v=0) in3=61(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 305001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=122(v=0) in1=109(v=1) in2=123(v=0) in3=61(v=1) || out0=123(v=0) out1=109(v=1)
# UVM_INFO router_driver.sv(70) @ 315000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=226(v=1) in1=3(v=1) in2=177(v=0) in3=185(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 315001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=226(v=1) in1=3(v=1) in2=177(v=0) in3=185(v=0) || out0=226(v=1) out1=3(v=1)
# UVM_INFO router_driver.sv(70) @ 325000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=5(v=1) in1=48(v=1) in2=24(v=1) in3=201(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 325001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=5(v=1) in1=48(v=1) in2=24(v=1) in3=201(v=1) || out0=5(v=1) out1=48(v=1)
# UVM_INFO router_driver.sv(70) @ 335000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=213(v=0) in1=103(v=1) in2=223(v=0) in3=115(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 335001: uvm_test_top.env.agent_monitor [router_monitor] Observed transaction: in0=213(v=0) in1=103(v=1) in2=223(v=0) in3=115(v=1) || out0=223(v=0) out1=103(v=1)
# UVM_INFO router_driver.sv(70) @ 345000: uvm_test_top.env.agent_driver [router_driver] Driving transaction: in0=130(v=1) in1=137(v=0) in2=45(v=1) in3=161(v=1) || out0=0(v=0) out1=0(v=0)
```

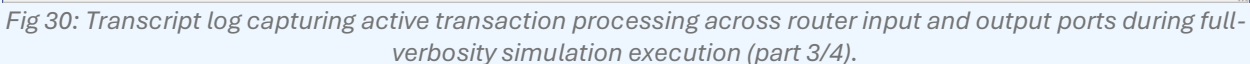
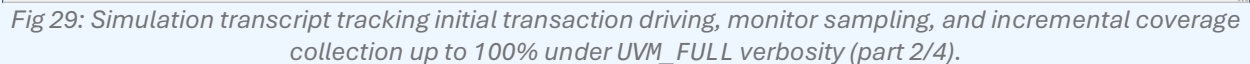
Fig 26: Transcript capturing driver and monitor transaction execution under UVM_HIGH filtering (part 2/3).

```
Transcript
File Edit View Bookmarks Window Help
Transcript
# UVM_INFO router_monitor.sv(53) @ 285001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=175(v=1) in1=209(v=0) in2=128(v=1) in3=74(v=1) || out0=175(v=1) out1=74(v=1)
# UVM_INFO router_driver.sv(70) @ 295000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=44(v=1) in1=87(v=1) in2=45(v=1) in3=211(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 295001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=44(v=1) in1=87(v=1) in2=45(v=1) in3=211(v=1) || out0=44(v=1) out1=87(v=1)
# UVM_INFO router_driver.sv(70) @ 305000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=122(v=0) in1=109(v=1) in2=123(v=0) in3=61(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 305001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=122(v=0) in1=109(v=1) in2=123(v=0) in3=61(v=1) || out0=123(v=0) out1=109(v=1)
# UVM_INFO router_driver.sv(70) @ 315000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=226(v=1) in1=3(v=1) in2=177(v=0) in3=189(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 315001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=226(v=1) in1=3(v=1) in2=177(v=0) in3=189(v=0) || out0=226(v=1) out1=3(v=1)
# UVM_INFO router_driver.sv(70) @ 325000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=5(v=1) in1=48(v=1) in2=24(v=1) in3=201(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 325001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=5(v=1) in1=48(v=1) in2=24(v=1) in3=201(v=1) || out0=5(v=1) out1=48(v=1)
# UVM_INFO router_driver.sv(70) @ 335000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=213(v=0) in1=103(v=1) in2=223(v=0) in3=115(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 335001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=213(v=0) in1=103(v=1) in2=223(v=0) in3=115(v=1) || out0=223(v=0) out1=103(v=1)
# UVM_INFO router_driver.sv(70) @ 345000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=130(v=1) in1=137(v=0) in2=45(v=1) in3=161(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 345001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=130(v=1) in1=137(v=0) in2=45(v=1) in3=161(v=1) || out0=130(v=1) out1=161(v=1)
# UVM_INFO router_driver.sv(70) @ 355000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=30(v=1) in1=93(v=1) in2=17(v=1) in3=49(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 355001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=30(v=1) in1=93(v=1) in2=17(v=1) in3=49(v=1) || out0=30(v=1) out1=93(v=1)
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1268) @ 355020: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO router_coverage.sv(76) @ 355020: uvm_test_top.env.coverage [router_coverage] Final functional coverage = 100.00%
# UVM_INFO router_scoreboard.sv(76) @ 355020: uvm_test_top.env.scoreboard [router_scoreboard] Scoreboard summary: 33 cycles checked, 0 errors
--- UVM Report Summary ---
** Report counts by severity
UVM_INFO : 97
UVM_WARNING : 0
UVM_ERROR : 0
UVM_FATAL : 0
** Report counts by id
[Questas UVM] 3
[RNTST] 1
[TEST_DONE] 1
[router_agent] 3
[router_base_sequence] 1
[router_base_test] 3
[router_coverage] 4
[router_driver] 35
[router_env] 3
[router_monitor] 36
[router_scoreboard] 4
[router_sequencer] 3
** Note: tfinish : C:/questasim64_10.6c/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
Time: 355020 ps Iteration: 54 Instance: /rb_top
# Break in Task uvm_pkg/uvm_root::run_test at C:/questasim64_10.6c/win64/.../verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
# A time value could not be extracted from the current line
VSIEM 4>
```

Fig 27: Final simulation summary under UVM_HIGH showing clean test completion, zero errors, and final report counts (part 3/3).

```
Transcript
File Edit View Bookmarks Window Help
Transcript
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(215) @ 0: reporter [Questas UVM] QUESTA_UVM-1.2.3
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(217) @ 0: reporter [Questas UVM] questa_uvm::init(+struct)
# UVM_INFO @ 0: reporter [RNTST] Running test router_base_test...
# UVM_INFO router_test.sv(19) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_LOW
# UVM_INFO router_test.sv(20) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_MEDIUM
# UVM_INFO router_test.sv(21) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_HIGH
# UVM_INFO router_test.sv(22) @ 0: uvm_test_top [router_base_test] [router_base_test] build_phase @ UVM_FULL
# UVM_INFO router_env.sv(21) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_LOW
# UVM_INFO router_env.sv(22) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_MEDIUM
# UVM_INFO router_env.sv(23) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_HIGH
# UVM_INFO router_env.sv(24) @ 0: uvm_test_top.env [router_env] [router_env] build_phase @ UVM_FULL
# UVM_INFO router_agent.sv(20) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_LOW
# UVM_INFO router_agent.sv(21) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_MEDIUM
# UVM_INFO router_agent.sv(22) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_HIGH
# UVM_INFO router_agent.sv(23) @ 0: uvm_test_top.env.agent [router_agent] [router_agent] build_phase @ UVM_FULL
# UVM_INFO router_driver.sv(24) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_LOW
# UVM_INFO router_driver.sv(25) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_MEDIUM
# UVM_INFO router_driver.sv(26) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_HIGH
# UVM_INFO router_driver.sv(27) @ 0: uvm_test_top.env.agent.driver [router_driver] [router_driver] build_phase @ UVM_FULL
# UVM_INFO router_monitor.sv(23) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_LOW
# UVM_INFO router_monitor.sv(24) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_MEDIUM
# UVM_INFO router_monitor.sv(25) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_HIGH
# UVM_INFO router_monitor.sv(26) @ 0: uvm_test_top.env.agent.monitor [router_monitor] [router_monitor] build_phase @ UVM_FULL
# UVM_INFO router_sequencer.sv(13) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_LOW
# UVM_INFO router_sequencer.sv(14) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_MEDIUM
# UVM_INFO router_sequencer.sv(15) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_HIGH
# UVM_INFO router_sequencer.sv(16) @ 0: uvm_test_top.env.agent.sequencer [router_sequencer] [router_sequencer] build_phase @ UVM_FULL
# UVM_INFO router_coverage.sv(46) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_LOW
# UVM_INFO router_coverage.sv(47) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_MEDIUM
# UVM_INFO router_coverage.sv(48) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_HIGH
# UVM_INFO router_coverage.sv(49) @ 0: uvm_test_top.env.coverage [router_coverage] [router_coverage] build_phase @ UVM_FULL
# UVM_INFO router_scoreboard.sv(25) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_LOW
# UVM_INFO router_scoreboard.sv(26) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_MEDIUM
# UVM_INFO router_scoreboard.sv(27) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_HIGH
# UVM_INFO router_scoreboard.sv(28) @ 0: uvm_test_top.env.scoreboard [router_scoreboard] [router_scoreboard] build_phase @ UVM_FULL
# UVM_INFO verilog_src/questa_uvm_pkg-1.2/src/questa_uvm_pkg.sv(277) @ 0: reporter [Questas UVM] End Of Elaboration
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'run'
# UVM_INFO router_sequencer.sv(21) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] Starting router_base_sequence with 30 transactions
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'pre_reset'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'reset'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'post_reset'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'pre_configure'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'configure'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'post_configure'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'pre_main'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'main'
# UVM_INFO verilog_src/uvm-1.1d/src/seq/uvm_sequencer_base.svh(1385) @ 0: uvm_test_top.env.agent.sequencer [PHASESEQ] No default phase sequence for phase 'post_main'
```

Fig 28: QuestaSim transcript log showing testbench component elaboration and UVM phase execution initialization (UVM_LOW through UVM_FULL) for router_base_test (part 1/4).



```
Transcript
File Edit View Bookmarks Window Help

# UVM_INFO router_driver.sv(70) @ 315000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=226(v=1) in1=3(v=1) in2=177(v=0) in3=189(v=0) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_coverage.sv(70) @ 315001: uvm_test_top.env.coverage [router_coverage] Sampled coverage, current cg_router=100.004
# UVM_INFO router_monitor.sv(53) @ 315001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=226(v=1) in1=3(v=1) in2=177(v=0) in3=189(v=0) || out0=226(v=1) out1=3(v=1)
# UVM_INFO router_driver.sv(70) @ 325000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=5(v=1) in1=48(v=1) in2=24(v=1) in3=201(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 325001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=5(v=1) in1=48(v=1) in2=24(v=1) in3=201(v=1) || out0=5(v=1) out1=48(v=1)
# UVM_INFO router_coverage.sv(70) @ 325001: uvm_test_top.env.coverage [router_coverage] Sampled coverage, current cg_router=100.004
# UVM_INFO router_driver.sv(70) @ 335000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=213(v=0) in1=103(v=1) in2=223(v=0) in3=115(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_coverage.sv(70) @ 335001: uvm_test_top.env.coverage [router_coverage] Sampled coverage, current cg_router=100.004
# UVM_INFO router_monitor.sv(53) @ 335001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=213(v=0) in1=103(v=1) in2=223(v=0) in3=115(v=1) || out0=223(v=0) out1=103(v=1)
# UVM_INFO router_driver.sv(70) @ 345000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=130(v=1) in1=137(v=0) in2=45(v=1) in3=161(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_monitor.sv(53) @ 345001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=130(v=1) in1=137(v=0) in2=45(v=1) in3=161(v=1) || out0=130(v=1) out1=161(v=1)
# UVM_INFO router_coverage.sv(70) @ 345001: uvm_test_top.env.coverage [router_coverage] Sampled coverage, current cg_router=100.004
# UVM_INFO router_driver.sv(70) @ 355000: uvm_test_top.env.agent.driver [router_driver] Driving transaction: in0=30(v=1) in1=93(v=1) in2=17(v=1) in3=49(v=1) || out0=0(v=0) out1=0(v=0)
# UVM_INFO router_coverage.sv(70) @ 355001: uvm_test_top.env.coverage [router_coverage] Sampled coverage, current cg_router=100.004
# UVM_INFO router_monitor.sv(53) @ 355001: uvm_test_top.env.agent.monitor [router_monitor] Observed transaction: in0=30(v=1) in1=93(v=1) in2=17(v=1) in3=49(v=1) || out0=30(v=1) out1=93(v=1)
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1245) @ 355020: reporter [TEST_DONE] All end-of-test objections have been dropped. Calling stop tasks
# UVM_INFO verilog_src/uvm-1.1d/src/base/uvm_objection.svh(1268) @ 355020: reporter [TEST_DONE] 'run' phase is ready to proceed to the 'extract' phase
# UVM_INFO router_coverage.sv(76) @ 355020: uvm_test_top.env.coverage [router_coverage] Final functional coverage = 100.004
# UVM_INFO router_scoreboard.sv(76) @ 355020: uvm_test_top.env.scoreboard [router_scoreboard] Scoreboard summary: 33 cycles checked, 0 errors

#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 152
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [PHASESEQ] 13
# [QueueTask UVM] 3
# [RNTST] 1
# [TEST_DONE] 2
# [router_agent] 4
# [router_base_sequence] 1
# [router_base_test] 4
# [router_coverage] 38
# [router_driver] 36
# [router_env] 4
# [router_monitor] 37
# [router_scoreboard] 5
# [router_sequence] 4
# ** Note: $finish : C:/questasim64_10.6c/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 355020 ps Iteration: 54 Instance: /tb_top
# 1
# Break in Task uvm_pkg::run_test at C:/questasim64_10.6c/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh line 430
v$SIM5>
```

Fig 31: Final transcript phase showing objection dropping, scoreboard completion check (33 cycles, 0 errors), 100% coverage summary, and test execution wrap-up (part 4/4).

BUG CHECK: ROUTER.V

Finding — dead clock/reset ports: router.v declares clk and rst_n as module inputs, but neither signal is referenced anywhere in the module body. The design is purely combinational (data_out0/data_out1/valid_out0/valid_out1 are driven entirely by assign statements off the data_in*/valid_in* inputs). Practical consequences:

- **No defined reset state.** There is no logic that responds to rst_n, so the outputs have no guaranteed value before the first valid input arrives — they simply reflect whatever the current combinational inputs happen to be, valid or not.
- **clk is unused.** For a module named "router" with a clock port, this is the kind of thing a lint pass (or -Wall in an RTL linter) would flag as a dead port — it strongly suggests the outputs were originally meant to be registered, or the ports were left over from an earlier synchronous version of the design.

Not corrected in the submitted RTL, on purpose: the testbench (driver, monitor, and scoreboard's reference model) was built against the DUT's actual behavior — same-cycle combinational response. Registering the outputs to "fix" the dead ports would silently change the DUT's protocol timing without any specification for what the correct reset value or added pipeline latency should be, which would go beyond what this assignment defines. The oversight is instead flagged here and directly in router.v as a comment, for the DUT owner to address.

Functional logic — verified correct. The routing/arbitration behavior itself (even-numbered inputs merging onto out0 with in0 priority over in2; odd-numbered inputs merging onto out1 with in1 priority over in3; valid_out asserting on either contributing valid_in) was independently re-derived in router_scoreboard.sv as a reference model and checked against the DUT every clock cycle. Across all four QuestaSim verbosity runs, the scoreboard reported:

Scoreboard summary: 33 cycles checked, 0 errors

with 0 mismatches in every run, including directed corner cases (all-inputs-idle, and both output channels' input pairs contending simultaneously). This confirms the arbitration/priority logic as written is functionally correct for the specification documented in the DUT's own header comment — the only issue found was the dead clk/rst_n ports noted above.